

COMPLETE WORKSHOP · LESSONS 1–8

OAuth 2.0, JWT/JOSE & FAPI 2.0

From the core mental model to high-security FAPI — one connected story.

Flows

Tokens

JWT / JOSE

Security controls

FAPI 2.0

Eight lessons, one mental model

- | | | |
|---|---|---|
| 1 | OAuth Problem, Actors & Endpoints | Why OAuth exists; the four actors; tokens & consent |
| 2 | Authorization Code + PKCE | The main user flow; front vs back channel; PKCE |
| 3 | Client Credentials, Refresh & Device | Pick the right flow for the app shape |
| 4 | Token Validation & JWT Basics | Decoding ≠ validation; the checklist |
| 5 | JWS, JWE, JWK / JWKS | Signing vs encryption; publishing & rotating keys |
| 6 | OAuth Security Controls | Attacks mapped to mitigations |
| 7 | Advanced: PAR, JAR, JARM, DPOP, mTLS | Harden request, response, token |
| 8 | FAPI 2.0 | Stricter OAuth for high-value APIs |

01

PART 01

OAuth Problem, Actors & Endpoints

OAuth lets a client get limited API access on a user's behalf — without ever receiving the password.

A framework, not a login or a token format

OAuth 2.0 IS

- An authorization framework (RFC 6749)
- Limited, scoped access via tokens
- The answer to: what may this client access?

OAuth 2.0 is NOT

- Password sharing
- One endpoint or token format
- A login protocol (that's OpenID Connect)
- Automatically secure

Framework = the model · RFCs = the spec · Flows = ways to use it.

Password sharing is the wrong shape

Give away the password

- Does everything you can
- No way to scope
- Revoke = change your password
- Breach exposes the whole account

Give a scoped token

- Limited to what's needed
- Password stays with the trusted service
- Revoke just this app, anytime
- Expires on its own

The client receives permission, not the user's password.

The four actors

Resource Owner

Owns the data — the user

Client

Wants access — the app

Authorization Server

Authenticates, gets consent, issues tokens

Resource Server

Hosts the API, validates tokens

Ask: who owns the data, who wants access, who grants it, who serves the API?

Two endpoints, two channels

Endpoint	Job	Channel
/authorize	User logs in & approves	Front-channel (browser) — exposed
/token	Exchange a grant for a token	Back-channel (server) — protected

Front-channel vs back-channel is the idea behind every later control.

The access token

Permission, not a password · scoped · expiring · revocable · checked on every call.

OAuth vs OpenID Connect

	OAuth 2.0	OpenID Connect
Purpose	Authorization	Authentication
Question	What can this client access?	Who is the user?
Main token	Access token	ID token
Example	Connect GitHub / Drive	“Log in with Google”

Call an API → OAuth. Know who you are → OpenID Connect.

PART 02

Authorization Code + PKCE

The browser carries only a short-lived code; tokens come from the back-channel.
PKCE protects the exchange.

Authorization Code, step by step

- 1 Client redirects the browser to /authorize
- 2 User logs in and approves
- 3 Server redirects back with code + state
- 4 Client posts the code to /token (back-channel)
- 5 Client receives the access token (+ refresh)
- 6 Client calls the API with the access token

Authorization request parameters

Parameter	Purpose
response_type=code	Ask for an authorization code
client_id	Identify the client
redirect_uri	Where the browser returns (must match)
scope	Permissions requested
state	Bind the callback to this client (CSRF)
code_challenge (S256)	PKCE hash for the code exchange

A stolen code should not be enough

Without PKCE

- Attacker intercepts the code
- Sends it to /token
- Public client → may get tokens

With PKCE

- Client sends code_challenge (a hash)
- Proves it with code_verifier at /token
- No verifier → attack fails

```
code_challenge = BASE64URL( SHA256(code_verifier) ) // S256, one-way
```

state vs redirect_uri vs PKCE

Check	Checked by	Protects against
state	Client	Callbacks the client didn't start (CSRF)
redirect_uri	Authorization server	Binds the code to the original target
code_verifier	Token endpoint	Proves the redeemer started the flow

Different problems — a modern browser flow uses all three.

03

PART 03

Client Credentials, Refresh & Device

OAuth is not one flow — different trust situations need different flows.

Three more flows, three problems

Client Credentials

Machine-to-machine. No user, no browser. The token represents the service.

Refresh Token

Renew a short-lived access token without sending the user through the flow again.

Device Authorization

CLI / TV: the user approves on a second device while the client polls.

Rotation makes reuse detectable

Step	What happens
Issue	Client gets access token AT1 + refresh token RT1
Refresh	AT1 expires; client sends RT1 to /token
Rotate	Server returns AT2 + RT2, marks RT1 used
Reuse seen	RT1 appears again → theft; revoke the family

An access token opens the door once; a refresh token keeps making keys.

Choose the flow

Flow	User?	Redirect?	Main use	Today?
Auth Code + PKCE	Yes	Yes	Web, SPA, mobile with a user	Recommended
Client Credentials	No	No	Service-to-service APIs	Recommended
Refresh Token	Sometimes	No	Renew an access token	With controls
Device Authorization	Yes	Partial	CLI / TV / limited input	When needed
Implicit / Password	Yes	—	Legacy	Avoid

PART 04

Token Validation & JWT Basics

A JWT is only a format. Security comes from validation — decoding is not validation.

JWT: header . payload . signature

decoded payload

```
{
  "iss": "https://auth.example.com",
  "sub": "user_123",
  "aud": "https://api.example.com",
  "exp": 1760000000,
  "scope": "repo.read repo.write"
}
```

Claims that drive validation

- iss — trust this issuer?
- aud — meant for this API?
- exp / nbf — valid right now?
- scope — enough permission?

A signed JWT is only base64url-encoded — anyone holding it can read the payload.

What the resource server must check

Validation checklist

- Signature verifies with a trusted key
- alg expected; kid is trusted
- iss trusted · aud is this API
- exp / nbf valid in time
- scope sufficient

Reject for the right reason

- 401 — missing / expired / bad token
- 403 — valid token, not enough scope
- Opaque (non-JWT) token → introspect

PART 05

JWS, JWE, JWK / JWKS

JWS signs, JWE encrypts, JWK/JWKS publish the keys. Signed does not mean private.

05

Five pieces that fit together

Term	Purpose
JWT	Compact claims format (the container)
JWS	Signature — integrity / authenticity
JWE	Encryption — confidentiality
JWK	JSON representation of one key
JWKS	A published set of keys (selected by kid)

JWS proves the content wasn't modified. JWE hides the content.

Where the verifier gets the key

How verification works

- AS signs with its private key
- AS publishes the public key in JWKS
- RS fetches JWKS from a trusted uri
- Selects by kid, then verifies

Common alg / kid mistakes

- Accepting alg=none
- Trusting the token's algorithm
- Keys from a token-controlled URL
- Dropping an old key too early

Allowed algorithms and key sources come from configuration — never the token.

PART 06

OAuth Security Controls

Connect common attacks to the controls that stop them.

06

Avoid Implicit and Password grant

Implicit flow

- Tokens returned in the browser redirect
- Front-channel exposure (history, logs)
- Replaced by Auth Code + PKCE

Password credentials

- Client collects the user's real password
- Breaks the core OAuth idea
- Blocks MFA; a breach exposes creds

Auth Code + PKCE for users · Client Credentials for services.

Attacks mapped to mitigations

Attack / mistake	Mitigation
Missing state	Generate, store, and verify state
Weak redirect URI matching	Exact, pre-registered redirect URIs
Authorization code interception	PKCE + short-lived single-use codes
Long-lived / stolen access token	Short token lifetime
Refresh token theft	Rotation + reuse detection
Bearer token replay	Sender-constrained tokens (DPoP / mTLS)

07

PART 07

Advanced: PAR, JAR, JARM, DPoP, mTLS

Harden the request, the response, and proof of token possession.

PAR, JAR, JARM

Control	What it does	Protects
PAR	Push request details to a back-channel endpoint	Request exposure in the browser
JAR	Sign the authorization request (a JWT)	Request-parameter integrity
JARM	Sign the authorization response (a JWT)	Response integrity, mix-up

PAR pushes the request · JAR signs the request · JARM signs the response.

DPoP vs mTLS

DPoP

- App-layer signed proof per request
- Token bound to a public key
- Fits public clients & APIs

mTLS

- TLS-layer client certificate
- Certificate-bound access tokens
- Fits confidential / high-security clients

Both sender-constrain the token: a stolen bearer token alone is not enough.

PART 08

FAPI 2.0

A high-security profile — stricter OAuth for high-value APIs, not a replacement protocol.

08

Stricter OAuth, combined

FAPI 2.0 combines

- Strict OAuth configuration
- PAR + sender-constrained tokens
- Strong client authentication
- Strict token validation

Where it's used

- Open banking & payments
- Healthcare & sensitive identity
- Regulated, high-impact APIs

Normal OAuth gives options; FAPI chooses the strict ones.

FAPI controls mapped to risk

Risk	FAPI-oriented control
Authorization request tampering	PAR + signed request objects (JAR)
Bearer token replay	Sender-constrained tokens (DPoP / mTLS)
Weak client authentication	Private-key or certificate-based auth
Token accepted by the wrong API	Strict audience validation
Payment / consent dispute	Message signing + audit evidence

YOU CAN TEACH THIS WHEN...

Ready to teach

- ✓ Draw Auth Code + PKCE and Client Credentials from memory
- ✓ Explain code vs access vs refresh vs ID token
- ✓ List the JWT validation checks (decoding isn't validation)
- ✓ Connect attacks to controls: PKCE, state, redirect, DPoP/mTLS
- ✓ Explain FAPI as strict OAuth for high-value APIs